

Project Title: Machine Learning Classifier Code

This code compares the performance of three different machine learning classifiers: Random Forest, Logistic Regression, and a Multi-layer Perceptron (MLP) Neural Network on a supervised learning task.

Methodology

The data was split into training (X_train, y_train) and testing (X_test, y_test) sets. Each model was trained on the complete feature set of the training data and evaluated using the accuracy metric on the unseen test data. I varied the iterations on the logistic regression and MLP neural network with similar accuracy results obtained.

Classifier	Configuration	Purpose
Random Forest	n_estimators=100, random_state=42	Ensemble of decision trees for robust classification.
Logistic Regression	max_iter=1200	Linear classification model used as a baseline.
MLP Classifier	max_iter=1200	A simple neural network to explore non-linear capabilities.

Results and Conclusion

The MLP Classifier achieved the highest accuracy, closely followed by the Random Forest model.

Classifier	Accuracy
MLP (Neural Network)	97.58%
Random Forest	96.75%
Logistic Regression	92.02%

Conclusion: Both the Random Forest and the MLP models demonstrated superior performance compared to the linear Logistic Regression baseline. The MLP Classifier provided the best generalization capability on the test dataset.

```
Part1 : In this section, we want to classify the MNIST dataset using Random Forest, Logistic Regression, and Neural Networks. We will print the accuracy for each model. The code for Random Forest will serve as an example, and you should complete the code for Logistic Regression and Neural Networks accordingly.

[20] ✓ 0s from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

[30] ✓ 0s # Train Different Classifiers

# 1. Random Forest
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=0) # Initialize the Random Forest model.
rf_classifier.fit(X_train, y_train) # Train the model on the training data.
y_pred_rf = rf_classifier.predict(X_test) # Make predictions on the test set.
accuracy_rf = accuracy_score(y_test, y_pred_rf) # Calculate the model's accuracy.

[50] ✓ 0s # 2. Logistic Regression
log_regresor = LogisticRegression(max_iter=1000) # Initialize the Logistic Regression model.
log_regresor.fit(X_train, y_train) # Train the model on the training data.
y_pred_logistic = log_regresor.predict(X_test) # Make predictions on the test set.
accuracy_logistic = accuracy_score(y_test, y_pred_logistic) # Calculate the model's accuracy.

[20] ✓ 0s # 3. MLP (Neural Network)
mlp = MLPClassifier(max_iter=1000) # Train the MLP
mlp.fit(X_train, y_train) # Train the MLP
y_pred_mlp = mlp.predict(X_test) # Make predictions on the test set.
accuracy_mlp = accuracy_score(y_test, y_pred_mlp) # Calculate the model's accuracy.

[30] ✓ 0s # Step 6: Print Accuracy Results
print("Logistic Regression Accuracy: {accuracy_logistic * 100:.2f}%")
print("Random Forest Accuracy: {accuracy_rf * 100:.2f}%")
print("MLP Accuracy: {accuracy_mlp * 100:.2f}%")

Logistic Regression Accuracy: 92.02%
Random Forest Accuracy: 96.75%
MLP Accuracy: 87.58%
```

📷YOLOv5 Face Detection Readme

This script utilizes the **YOLOv5s** model for object detection, specifically to detect faces within an image.

Goal: To visually confirm that the YOLOv5s model successfully identifies and localizes objects (faces) in the input image.

🔧 Execution Steps

The code performs the following actions:

1. **Model Loading:** The pre-trained YOLOv5 model is loaded.
2. **Image Input:** The input image, named **tofunmi.jpg**, is specified. This image is accessible within the YOLOv5 file pathway.
3. **Inference:** The model runs **inference** (`results = model(img)`), processing the image to identify and localize objects (faces).
4. **Visualization:** The results are displayed (`results.show()`), showing the original image with bounding boxes drawn around the detected objects.

Conclusion: The image detection is successfully executed and detected the loaded picture as a person with 85% accuracy. It also detects partially some other objects in the background which are a desktop screen (identified as laptop – 40%), a shape (identified as a bird – 26%) on the patterned shirt in the image.

